Name: Jibika Gautam

Q1. Write a C program to find the maximum and minimum elements in an array.

```
#include <stdio.h>



int main() {

    int n, min, max;

    printf("Enter the size of array.");

    scanf("%d", &n);



    int arr[n];

    printf("Enter elements of array: ");

    for (int i = 0; i < n; i++){

        scanf("%d", &arr[i]);

    }



    min = max = arr[0];

    for(int i = 1; i<n; i++){

        if(arr[i] < min) min =arr[i];

        if (arr[i] > max ) max =arr[i];

    }

    printf("Minimum: %d\n", min);

    printf("Maximum: %d\n", max);
```

```
    return 0;

}
```

```
Enter the size of array.3
Enter elements of array: 2 6 3
Minimum: 2
Maximum: 6
```

Q2. Implement a C program to reverse the elements of an array.

```
#include <stdio.h>

int main(){



int n, temp;

printf("Enter the size of array: ");

scanf("%d",&n);



int arr[n];

printf("Enter elements of array: ");

for(int i=0; i<n; i++)

scanf("%d", &arr[i]);



printf("The element of array are: ");

for(int i=0; i<n; i++)

printf("%d", arr[i]);

printf("\n");
```

```
for(int i =0; i<n/2; i++){

    temp = arr[i];

    arr[i]= arr[n -1-i];

    arr[n-1-i] = temp;

}

printf("The elements of array after reversing are: ");

for(int i=0; i<n; i++)

printf("%d",arr[i]);



    return 0;

}
```

```
Enter the size of array: 4
Enter elements of array: 2 7 3 5
The element of array are: 2735
The elements of array after reversing are: 5372
```

Q3. Write a C program to sort an array of integers in ascending order using a sorting algorithm of your choice (e.g., bubble sort, selection sort, insertion sort).

```
#include<stdio.h>

int main (){

    int n, temp;

    printf("Enter the size of array: ");

    scanf("%d",&n);



    int arr[n];
```

```
    printf(" the elements of array: ");

    for( int i=0; i<n; i++)

    scanf("%d",&arr[i]);

    printf("\n");



    for(int i=0; i<n; i++)

    {

        for(int j=0; j <n-1-i; j++)

        {

            if(arr[j] > arr[j + 1])

            {

                temp=arr[j];

                arr[j] = arr[j + 1];

                arr[j + 1]=temp;

            }

        }

    }

    printf("The elements of array after sort are: " );

    for(int i =0; i<n; i++)

    printf("%d", arr[i]);

    printf("\n");
```

```
    return 0;

}
```

```
Enter the size of array: 4
 the elements of array: 3 5 1 6

The elements of array after sort are: 1356
```

Q4. Implement a C program to find the second largest element in an array.

```
#include <stdio.h>



int main() {

    int n, temp;

    printf("Enter the size of array: ");

    scanf("%d", &n);



    if(n < 2) {

        printf("Array must have at least 2 elements.\n");

        return 1;

    }



    int arr[n];

    printf("Enter elements of array: ");

    for(int i = 0; i < n; i++)

        scanf("%d", &arr[i]);
```

```
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - 1 - i; j++) {
            if(arr[j] < arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    // Display sorted array
    printf("Sorted array (descending): ");
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    int secondLargest = -1;
    for(int i = 1; i < n; i++) {
        if(arr[i] != arr[0]) {
            secondLargest = arr[i];
            break;
        }
    }
```

```
    if(secondLargest == -1)

        printf("No distinct second largest element found.\n");

    else

        printf("Second largest element: %d\n", secondLargest);



    return 0;

}
```

```
Enter the size of array: 4
Enter elements of array: 3 6 2 5
Sorted array (descending): 6 5 3 2
Second largest element: 5
```

Q5. Write a C program to merge two sorted arrays into a single sorted array.

```
#include <stdio.h>

int main(){



    // [2,4,6]

    //[3,4,7,9]

    //[2,3,4,4,6,7,9]



    int n1, n2;



    printf("Enter the size of first and second array: ");

    scanf("%d %d", &n1, &n2 );
```

```
    int arr1[n1], arr2[n2];

    printf("Enter the value of first array: ");

    for(int i=0; i<n1; i++) scanf("%d", &arr1[i]);


    printf("Enter the value for second array: ");

    for(int j=0; j<n2; j++) scanf("%d", &arr2[j]);


    int merged_arr[n1 +n2];


    int x =0, z=0, y=0;


    while (x<n1 || y<n2) {

        if (arr1[x] < arr2[y]) merged_arr[z++] = arr1[x++];

        else merged_arr[z++] = arr2[y++];

    }


while (x<n1) merged_arr[z++] = arr1[x++];

while (y<n2) merged_arr[z++] = arr2[y++];


printf("Merged array: ");

for (int k=0; k<n1+n2; k++) printf("%d ", merged_arr[k]);

printf("\n");
```

```
    return 0;

}
```

```
Enter the size of first and second array: 3 4
Enter the value of first array: 4 5 6 7
Enter the value for second array: 3 8 2 5
Merged array: 4 5 6 7 3 8 2
```

Q6. Write a C program to find the length of a string without using the built-in string functions.

```
#include<stdio.h>

int main (){



    char abc[100];

    int count = 0;



    printf("Enter the string: ");

    scanf("%s",&abc );



    for (int i = 0; i<=100; i++){

        if (abc[i] == '\0'){

            break;

        }

        count++;

    }

    printf("Length of string is %d.\n", count);
```

```
    return 0;

}
```

```
Enter the string: jibika
Length of string is 6.
```

Q7. Implement a C program to reverse a string.

```
#include <stdio.h>



int main() {

    char str[100];

    char reversed[100];

    int length = 0, i, j;



    printf("Enter a string: ");

    scanf("%[^\n]s", str);



    for(i = 0; str[i] != '\0'; i++) {

        length++;

    }



    printf("Original string: %s\n", str);
```

```
    for(i = length - 1, j = 0; i >= 0; i--, j++) {

        reversed[j] = str[i];

    }

    reversed[length] = '\0';



    printf("Reversed string: %s\n", reversed);



    return 0;

}
```

```
Enter a string: rear
Original string: rear
Reversed string: raer
```

Q8. Write a C program to check if a given string is a palindrome.

```
#include <stdio.h>



int main() {

    char str[100];

    int length = 0, i, isPalindrome = 1;



    printf("Enter a string: ");

    scanf("%[^\n]s", str);



    for(i = 0; str[i] != '\0'; i++) {

        length++;
```

```
    }


    for(i = 0; i < length / 2; i++) {
        if(str[i] != str[length - 1 - i]) {
            isPalindrome = 0;
            break;
        }
    }


    printf("String: %s\n", str);


    if(isPalindrome) {
        printf("'%s' is a palindrome!\n", str);
    } else {
        printf("'%s' is not a palindrome.\n", str);
    }


    return 0;
}
```

```
Enter a string: madam
String: madam
'madam' is a palindrome!
```

Q9. Implement a C program to count the occurrence of a specific character in a string.

```
#include <stdio.h>
```

```c
int main() {
    char str[100];
    char searchChar;
    int count = 0;
    int i;

    printf("Enter a string: ");
    scanf("%[^\n]s", str);

    printf("Enter the character to count: ");
    scanf(" %c", &searchChar);
    for(i = 0; str[i] != '\0'; i++) {
        if(str[i] == searchChar) {
            count++;
        }
    }

    printf("\nString: %s\n", str);
    printf("Character '%c' appears %d time(s) in the string.\n",
searchChar, count);

    return 0;
}
```

```
Enter a string: nishant
Enter the character to count: h

String: nishant
Character 'h' appears 1 time(s) in the string.
```

Q10. Write a C program to concatenate two strings without using the built-in string functions.

```c
#include <stdio.h>



int main() {

    char str1[200], str2[100];

    int i, j;



    printf("Enter first string: ");

    scanf("%[^\n]s", str1);



    printf("Enter second string: ");

    scanf(" %[^\n]s", str2);



    i = 0;

    while(str1[i] != '\0') {

        i++;

    }



    j = 0;

    while(str2[j] != '\0') {
```

```
        str1[i] = str2[j];

        i++;

        j++;

    }

    str1[i] = '\0';



    printf("\nConcatenated string: %s\n", str1);



    return 0;

}
```

```
Enter first string: hello
Enter second string: darling

Concatenated string: hellodarling
```

Q11. Write a function named calculateAverage that takes an array of integers as input and returns the average of the numbers.

```
#include <stdio.h>



float calculateAverage(int *arr, int size) {

    int sum = 0;



    for(int i = 0; i < size; i++) {

        sum += *(arr + i);

    }
```

```
    return (float)sum / size;

}



int main() {

    int n;

    printf("Enter the size of array: ");

    scanf("%d", &n);



    int arr[n];

    printf("Enter %d elements: ", n);

    for(int i = 0; i < n; i++) {

        scanf("%d", &arr[i]);

    }



    float average = calculateAverage(arr, n);

    printf("\nAverage: %.2f\n", average);



    return 0;

}
```

```
Enter the size of array: 3
Enter 3 elements: 4 5 7

Average: 5.33
```

Q12. Write a function named isPalindrome that takes a string as input and returns 1 if it is a palindrome (reads the same forwards and backwards), and 0 otherwise.

```c
#include <stdio.h>

int isPalindromeRecursive(char str[], int left, int right) {

    if(left >= right) {

        return 1;

    }



    if(str[left] != str[right]) {

        return 0;

    }



    return isPalindromeRecursive(str, left + 1, right - 1);

}



int isPalindrome(char str[]) {

    int length = 0;



    // Find length

    while(str[length] != '\0') {

        length++;

    }



    return isPalindromeRecursive(str, 0, length - 1);

}
```

```c
int main() {
    char str[100];

    printf("Enter a string: ");
    scanf("%[^\n]s", str);

    if(isPalindrome(str)) {
        printf("\n'%s' IS a palindrome!\n", str);
    } else {
        printf("\n'%s' is NOT a palindrome.\n", str);
    }

    return 0;
}
```

```
Enter a string: warehouse

'warehouse' is NOT a palindrome.
```

Q13. Write a function named findFactorial that takes an integer as input and returns its factorial.

```c
#include<stdio.h>

int findFactorial(int num) {
```

```
    int i=1, fact=1;

    while (i<=num){

        fact*=i;

        i++;

    }

    return fact;

}



int main(){

    int num;

    printf("Enter the number: ");

    scanf("%d", &num);



    int factorial = findFactorial(num);

    printf("The factorial is %d.\n", factorial);



    return 0;

}
```

```
Enter the number: 4
The factorial is 24.
```

Q14. Write a function named convertTemperature that takes a temperature value in Celsius and converts it to Fahrenheit. The function should return the converted temperature.

```
#include <stdio.h>
```

```c
float convertTemperature(float celsius) {

    // Formula: Fahrenheit = (Celsius × 9/5) + 32

    return (celsius * 9.0 / 5.0) + 32;

}



int main() {

    float celsius, fahrenheit;



    printf("Enter temperature in Celsius: ");

    scanf("%f", &celsius);



    fahrenheit = convertTemperature(celsius);



    printf("\n%.2f° Celsius = %.2f° Fahrenheit\n", celsius, fahrenheit);



    return 0;

}
```

```
Enter temperature in Celsius: 100

100.00╥╗ Celsius = 212.00╥╗ Fahrenheit
```

Q15. Write a function named countOccurrences that takes a string and a character as input and returns the number of times the character appears in the string.

```c
#include <stdio.h>
```

```c
int countOccurrences(char str[], char ch) {

    int count = 0;



    for(int i = 0; str[i] != '\0'; i++) {

        if(str[i] == ch) {

            count++;

        }

    }



    return count;

}



int main() {

    char str[100];

    char ch;



    printf("Enter a string: ");

    scanf("%[^\n]s", str);



    printf("Enter the character to count: ");

    scanf(" %c", &ch);



    int result = countOccurrences(str, ch);
```

```
    printf("\nString: \"%s\"\n", str);

    printf("Character '%c' appears %d time(s)\n", ch, result);



    return 0;

}
```

```
Enter a string: racecar
Enter the character to count: r

String: "racecar"
Character 'r' appears 2 time(s)
```

Q16. Write a function named reverseArray that takes an array of integers as input and reverses the order of the elements in the array.

```
#include <stdio.h>



void reverseArray(int arr[], int size) {

    int temp;



    for(int i = 0; i < size / 2; i++) {

        temp = arr[i];

        arr[i] = arr[size - 1 - i];

        arr[size - 1 - i] = temp;

    }

}
```

```c
int main() {

    int n;



    printf("Enter array size: ");

    scanf("%d", &n);



    int arr[n];

    printf("Enter %d numbers: ", n);

    for(int i = 0; i < n; i++) {

        scanf("%d", &arr[i]);

    }



    printf("Original: ");

    for(int i = 0; i < n; i++) printf("%d ", arr[i]);



    reverseArray(arr, n);



    printf("\nReversed: ");

    for(int i = 0; i < n; i++) printf("%d ", arr[i]);

    printf("\n");



    return 0;

}
```

```
Enter array size: 3
Enter 3 numbers: 4 7 2
Original: 4 7 2
Reversed: 2 7 4
```

Q17. Write a function named calculatePower that takes two integers, base and exponent, as input and returns the result of raising the base to the exponent.

```c
#include<stdio.h>



int calculatePower( int base, int expodent){

    int result =1;

    for (int i =0; i<expodent; i++){

        result *=base;

    }

    return result;

}

int main(){

    int b,e;

    printf("Enter base and experiment: ");

    scanf("%d %d", &b,&e);



    int power = calculatePower(b,e);

    printf("Result is : %d.\n", power);



    return 0;

}
```

```
Enter base and experiment: 4 2
Result is : 16.
```

Q18. Write a function named findPrimeNumbers that takes an integer n as input and prints all prime numbers from 1 to n.

```c
#include <stdio.h>


int isPrime(int num) {

    if(num <= 1) return 0;

    if(num == 2) return 1;

    if(num % 2 == 0) return 0;


    for(int i = 3; i * i <= num; i += 2) {

        if(num % i == 0) return 0;

    }

    return 0;

}


void findPrimeNumbers(int n) {

    printf("Prime numbers from 1 to %d: ", n);


    for(int i = 2; i <= n; i++) {

        if(isPrime(i)) {

            printf("%d ", i);

        }
```

```
    }

    printf("\n");

}


int main() {

    int n;


    printf("Enter a number: ");

    scanf("%d", &n);


    if(n < 2) {

        printf("No prime numbers in range 1 to %d\n", n);

    } else {

        findPrimeNumbers(n);

    }


    return 0;

}
```

```
Enter a number: 4
Prime numbers from 1 to 4: 2
```

Q19. Write a function named calculateFactorialSeries that takes an integer n as input and prints the factorial series up to n.

```
#include <stdio.h>
```

```c
void calculateFactorialSeries(int n) {
    if (n < 1) {
        printf("Please enter a positive integer.\n");
        return;
    }

    long long fact = 1;

    printf("Factorial series up to %d:\n", n);

    for (int i = 1; i <= n; i++) {
        fact = fact * i;
        printf("%d! = %lld\n", i, fact);
    }
}

int main() {
    int n;

    printf("Enter a positive integer n: ");
    scanf("%d", &n);

    calculateFactorialSeries(n);
```

```
    return 0;

}
```

```
Enter a positive integer n: 4
Factorial series up to 4:
1! = 1
2! = 2
3! = 6
4! = 24
```

Q20. Write a function named calculateGCD that takes two integers as input and returns their greatest common divisor (GCD).

```
#include <stdio.h>



int calculateGCD(int a, int b) {

    a = (a < 0) ? -a : a;

    b = (b < 0) ? -b : b;



    while (b != 0) {

        int temp = b;

        b = a % b;

        a = temp;

    }

    return a;

}
```

```c
int main() {

    int num1, num2;



    printf("Enter two integers: ");

    scanf("%d %d", &num1, &num2);



    int gcd = calculateGCD(num1, num2);



    printf("GCD of %d and %d is %d\n", num1, num2, gcd);



    return 0;

}
```

```
Enter two integers: 3 5
GCD of 3 and 5 is 1
```

Q21. Write a recursive function named calculateFactorial that takes an integer n as input and returns its factorial.

```c
#include <stdio.h>



long long calculateFactorial(int n) {

    if (n == 0 || n == 1) {

        return 1;

    }



    return n * calculateFactorial(n - 1);
```

```c
}


int main() {
    int num;

    printf("Enter a non-negative integer: ");
    scanf("%d", &num);

    if (num < 0) {
        printf("Error: Factorial is not defined for negative numbers.\n");
        return 1;
    }

    long long result = calculateFactorial(num);

    printf("Factorial of %d is %lld\n", num, result);

    return 0;
}
```

```
Enter a non-negative integer: 5
Factorial of 5 is 120
```

Q22. Write a recursive function named calculateFibonacci that takes an integer n as input and returns the nth Fibonacci number. The Fibonacci sequence starts with 0 and 1, and each subsequent number is the sum of the two preceding numbers.

```c
#include <stdio.h>


long long calculateFibonacci(int n) {
    if (n == 0) {
        return 0;
    }
    if (n == 1) {
        return 1;
    }

    return calculateFibonacci(n - 1) + calculateFibonacci(n - 2);
}

int main() {
    int n;

    printf("Enter the position n (n >= 0): ");
    scanf("%d", &n);

    if (n < 0) {
        printf("Error: Please enter a non-negative integer.\n");
        return 1;
    }
```

```
    long long result = calculateFibonacci(n);


    printf("The %dth Fibonacci number is %lld\n", n, result);


    return 0;

}
```

```
Enter the position n (n >= 0): 5
The 5th Fibonacci number is 5
```

Q23. Write a recursive function named calculateGCD that takes two integers a and b as input and returns their greatest common divisor (GCD).

```
#include <stdio.h>


int calculateGCD(int a, int b) {

    a = (a < 0) ? -a : a;

    b = (b < 0) ? -b : b;


    if (b == 0) {

        return a;

    }


    return calculateGCD(b, a % b);

}
```

```c
int main() {

    int num1, num2;



    printf("Enter two integers: ");

    scanf("%d %d", &num1, &num2);



    int gcd = calculateGCD(num1, num2);



    printf("GCD of %d and %d is %d\n", num1, num2, gcd);



    return 0;

}
```

```
Enter two integers: 3 6
GCD of 3 and 6 is 3
```

Q24. Write a recursive function named calculatePower that takes two integers base and exponent as input and returns the result of raising the base to the exponent.

```c
#include <stdio.h>



long long calculatePower(int base, int exponent) {

    if (exponent == 0) {

        return 1;

    }



    if (exponent < 0) {
```

```
        printf("Error: Negative exponents are not supported.\n");
        return -1;
    }

    return base * calculatePower(base, exponent - 1);
}

int main() {
    int base, exponent;

    printf("Enter base and exponent: ");
    scanf("%d %d", &base, &exponent);

    if (exponent < 0) {
        printf("Negative exponents are not supported in this version.\n");
        return 1;
    }

    long long result = calculatePower(base, exponent);

    printf("%d raised to the power %d is %lld\n", base, exponent, result);

    return 0;
```

```
}
```

```
Enter base and exponent: 4 5
4 raised to the power 5 is 1024
```

Q25. Write a recursive function named calculateSumOfDigits that takes an integer n as input and returns the sum of its digits.

```
#include <stdio.h>



int calculateSumOfDigits(int n) {

    if (n == 0) {

        return 0;

    }



    if (n < 0) {

        n = -n;

    }



    return (n % 10) + calculateSumOfDigits(n / 10);

}



int main() {

    int num;



    printf("Enter a number: ");

    scanf("%d", &num);
```

```
    int sum = calculateSumOfDigits(num);

    printf("Sum of digits of %d is %d\n", num, sum);

    return 0;
}
```

```
Enter a number: 45
Sum of digits of 45 is 9
```

Q26. Write a recursive function named reverseString that takes a string as input and returns the reversed string.

```
#include <stdio.h>
#include <string.h>

void reverseString(char str[], int start, int end) {
    if (start >= end) {
        return;
    }

    char temp = str[start];
    str[start] = str[end];
    str[end] = temp;

    reverseString(str, start + 1, end - 1);
```

```
}


int main() {

    char str[100];


    printf("Enter a string: ");

    scanf("%s", str);

    int length = strlen(str);


    reverseString(str, 0, length - 1);


    printf("Reversed string: %s\n", str);


    return 0;

}
```

```
Enter a string: hell
Reversed string: lleh
```

Q27. Write a recursive function named printTriangle that takes an integer n as input and prints a triangle of asterisks (*) with n rows.

```
#include <stdio.h>


void printTriangle(int n) {

    if (n == 0) {

        return;
```

```
    }

    printTriangle(n - 1);

    for (int i = 0; i < n; i++) {
        printf("*");
    }
    printf("\n");
}

int main() {
    int rows;

    printf("Enter the number of rows: ");
    scanf("%d", &rows);

    if (rows < 1) {
        printf("Please enter a positive integer.\n");
        return 1;
    }

    printf("Triangle with %d rows:\n", rows);
    printTriangle(rows);
```

```
    return 0;

}
```

```
Enter the number of rows: 4
Triangle with 4 rows:
*
**
***
****
```

Q28. Write a recursive function named calculateBinary that takes an integer n as input and returns its binary representation as a string.

```
#include <stdio.h>



void calculateBinary(int n) {

    if (n == 0) {

        return;

    }

    calculateBinary(n / 2);

    printf("%d", n % 2);

}



int main() {

    int num;



    printf("Enter a non-negative integer: ");
```

```
    scanf("%d", &num);


    if (num < 0) {
        printf("Negative numbers are not supported.\n");
        return 1;
    }


    printf("Binary representation of %d is: ", num);


    if (num == 0) {
        printf("0");
    } else {
        calculateBinary(num);
    }


    printf("\n");


    return 0;
}
```

```
Enter a non-negative integer: 4
Binary representation of 4 is: 100
```

Q29. Write a recursive function named isPalindrome that takes a string as input and returns 1 if it is a palindrome (reads the same forwards and backwards), and 0 otherwise.

```
#include <stdio.h>
```

```c
int isPalindrome(char str[], int start, int end) {

    if (start >= end) {

        return 1;

    }



    if (str[start] != str[end]) {

        return 0;

    }



    return isPalindrome(str, start + 1, end - 1);

}



int main() {

    char str[100];



    printf("Enter a string: ");

    scanf("%s", str);



    int length = 0;

    while (str[length] != '\0') {

        length++;

    }
```

```
    int result = isPalindrome(str, 0, length - 1);



    if (result == 1) {

        printf("%s is a palindrome.\n", str);

    } else {

        printf("%s is not a palindrome.\n", str);

    }



    return 0;

}
```

```
Enter a string: racecar
racecar is a palindrome.
```

Q30. Write a recursive function named countOccurrences that takes a string and a character as input and returns the number of times the character appears in the string.

```
#include <stdio.h>
int countOccurrences(char str[], char ch, int index) {
    if (str[index] == '\0') {
        return 0;
    }

    int count = 0;
    if (str[index] == ch) {
        count = 1;
    }
    return count + countOccurrences(str, ch, index + 1);
}

int main() {
    char str[100];
    char ch;
```

```
    printf("Enter a string: ");
    scanf("%s", str);

    printf("Enter the character to count: ");
    scanf(" %c", &ch);

    int result = countOccurrences(str, ch, 0);

    printf("The character '%c' appears %d times in \"%s\".\n", ch, result,
str);

    return 0;
}
```

```
Enter a string: heel
Enter the character to count: e
The character 'e' appears 2 times in "heel".
```